МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

БРЯНСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Кафедра «Информатика и программное обеспечение»

КУРСОВОЙ ПРОЕКТ

Разработка и исследование сверточной нейронной сети для классификации изображений дорожных знаков

Всего листов 24

Преподаватель:

_____ #########

«___» _____ 2026 г.

Студент гр. О-24-ИВТ-1-ПО-Б

_____ ##########

«___» _____ 2026 г.

БРЯНСК 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ **4**

1.1. Задача классификации **4**

1.2. Исторический контекст **4**

1.3. Основные понятия и определения **5**

1.4. Типовые подходы к решению **6**

1.4.1. Обучение с нуля и Transfer Learning 6

1.4.2. Аугментация данных 6

1.4.3. Регуляризация и планирование learning rate 6

2. КОНСТРУКТОРСКАЯ ЧАСТЬ **7**

2.1. Архитектура модели **7**

2.2. Подготовка данных **9**

2.3. Алгоритм обучения **10**

2.3.1. Функция потерь и оптимизатор 10

2.3.2. Планировщик learning rate 11

2.3.3. Обратное распространение ошибки 11

2.4. Исследование устойчивости к искажениям **12**

2.5. Визуализация ошибок **13**

3. АНАЛИЗ ВЫЧИСЛИТЕЛЬНОЙ СЛОЖНОСТИ **15**

3.1. Теоретический подсчёт FLOPs **15**

3.2. Практическая оценка времени инференса **16**

3.3. Анализ потребления памяти **18**

4. ТЕСТИРОВАНИЕ **20**

5. ИТОГИ ПРОДЕЛАННОЙ РАБОТЫ **22**

СПИСОК ЛИТЕРАТУРЫ **24**

ВВЕДЕНИЕ

Актуальность. Распознавание дорожных знаков является одной из ключевых задач компьютерного зрения, имеющей непосредственное практическое применение в системах помощи водителю (ADAS) и автономных транспортных средствах. Несмотря на то что задача кажется хорошо изученной, она по-прежнему сложна в реальных условиях — при разном освещении, частичном перекрытии знаков, смазанности изображения и атмосферных помехах.

Современные сверточные нейронные сети демонстрируют высокую точность на стандартных бенчмарках, однако вопрос их устойчивости к различным искажениям, а также понимание причин ошибок остаются актуальными. Курсовой проект посвящён разработке и исследованию CNN для классификации изображений из датасета RTSD (Russian Traffic Sign Dataset) с акцентом на эмпирическую верификацию вычислительной сложности модели.

Цель работы — построить сверточную нейросеть, обучить её распознавать дорожные знаки по реальным фотографиям и провести исследование: как различные искажения (размытие, шум, частичное закрытие знака) и приёмы обучения (аугментации, регуляризация) влияют на точность, а также верифицировать соответствие теоретической вычислительной сложности реальным метрикам.

Задачи:

- разработать и обучить модель с нуля на готовом датасете
- провести эмпирическую верификацию теоретической вычислительной сложности и потребление памяти
- исследовать устойчивость модели к различным искажениям тестовых изображений
- применить методы интерпретируемости для анализа ошибок

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Задача классификации

Классификация изображений — одна из фундаментальных задач компьютерного зрения, цель которой состоит в автоматическом отнесении входного изображения к одному из заранее определённых классов. Задача возникает в самых разных прикладных областях: медицинская диагностика, контроль качества на производстве, распознавание объектов в робототехнике, анализ спутниковых снимков. Общая сложность задачи определяется числом классов, качеством и объёмом обучающей выборки, внутриклассовой изменчивостью (один и тот же объект при разном освещении, ракурсе, масштабе) и межклассовой схожестью (визуально похожие, но семантически разные категории).

В настоящей работе рассматривается частный случай этой задачи — автоматическое распознавание дорожных знаков в контексте интеллектуальных транспортных систем. Специфика данной постановки обусловлена значительным числом классов, сильным дисбалансом выборки (редкие знаки представлены единичными примерами), а также разнообразием условий съёмки: разное освещение, атмосферные помехи, частичное перекрытие знаков. В качестве датасета используется RTSD — набор реальных снимков дорожных знаков Российской Федерации, полученных с бортовых видеорегистраторов (очищенная версия: https://huggingface.co/datasets/eleldar/rtsd_cleaned).

1.2. Исторический контекст

Идея применения скользящего ядра свёртки к изображениям была формализована в работе LeCun et al. (1989) «Back-Propagation Applied to Handwritten Zip Code Recognition»[1]. Авторы показали, что разделение весов в сверточных слоях позволяет сети автоматически извлекать локальные

признаки например, края, текстуры, формы — при значительно меньшем числе параметров по сравнению с полносвязной архитектурой.

Работа Honglak Lee et al. (2011) «Unsupervised Learning of Hierarchical Representations with Convolutional Deep Belief Networks»[2] продемонстрировала, что послойная иерархия сверток обеспечивает выделение признаков возрастающей абстрактности: от краев и цветовых пятен в первых слоях до семантически значимых паттернов — в последних.

1.3. Основные понятия и определения

PyTorch — библиотека глубокого обучения с открытым исходным кодом, в составе имеет множества готовый инструментов нацеленных на создание нейросетей.

Тензор — базовая структура данных библиотеки PyTorch. N-мерный массив, обобщающий вектор и матрицу. Используется для хранения весов, промежуточных активаций и входных батчей. Форма батча: (N, C_{in}, H_{in}, W_{in}).

Вычислительный граф — направленный ациклический граф, автоматически строящийся PyTorch в процессе прямого прохода. Является основой механизма autograd и используется при обратном распространении ошибки.

Свёртка (слой Conv2d) — алгоритм скользящего окна: ядро K×K последовательно применяется ко всем пространственным позициям тензора. Ключевое свойство является разделение весов, резко сокращающее число параметров.

Batch Normalization — на этапе обучения вычисляет среднее и дисперсию по мини-батчу, нормализует активации и применяет аффинное преобразование с обучаемыми γ и β.

Max Pooling — субдискретизация скользящим окном: выбирает максимум в каждом регионе, уменьшая пространственное разрешение и добавляя инвариантность к малым сдвигам.

Dropout / Dropout2d — стохастическая регуляризация: случайно обнуляет нейроны (Dropout) или целые каналы (Dropout2d) с вероятностью p. В режиме работы сети автоматически отключается.

Grad-CAM и Saliency Map — алгоритмы интерпретируемости. Grad-CAM вычисляет градиенты скора относительно будущих карт последнего Conv-слоя и строит тепловую карту значимых регионов. Saliency Map показывает градиент скора по пикселям входного изображения.

1.4. Типовые подходы к решению

1.4.1. Обучение с нуля и Transfer Learning

В задачах компьютерного зрения применяются два основных подхода: обучение с нуля и перенос знаний из предобученной сети. В данном проекте выбран первый подход, поскольку цель не только получить высокую точность, но и провести детальный анализ вычислительной сложности собственной архитектуры.

1.4.2. Аугментация данных

Для повышения устойчивости модели применяются следующие виды аугментации:

- случайные повороты и горизонтальное отражение
- изменение яркости, контраста, насыщенности
- добавление гауссовского шума
- случайное размытие
- случайное закрытие фрагмента изображения

1.4.3. Регуляризация и планирование learning rate

Для предотвращения переобучения применяются Dropout и Dropout2d [3]. В качестве планировщика используется Cosine Annealing для изменение learning rate по косинусному закону, что позволяет точнее сойтись в конце обучения[4].

2. КОНСТРУКТОРСКАЯ ЧАСТЬ

2.1. Архитектура модели

Разработанная модель представляет собой трёхблочную сверточную нейронную сеть, архитектура которой построена по принципу постепенного увеличения числа каналов при одновременном уменьшении пространственного разрешения карт признаков. Входное изображение имеет размерность 3×48×48 (три цветовых канала RGB и разрешение 48×48 пикселей).

Каждый из трёх блоков содержит пару сверточных слоёв с ядром 3×3 и отступом в единицу, что сохраняет пространственный размер карты признаков внутри блока. После каждого сверточного слоя следует BatchNorm и нелинейная активация ReLU. Завершает блок MaxPool2d с шагом 2, вдвое уменьшающий пространственное разрешение, и Dropout2d с вероятностью 25%, случайно обнуляющий целые каналы для регуляризации. Число каналов последовательно удваивается от блока к блоку: 3→32→64→128. Такая схема позволяет первым слоям обнаруживать простые низкоуровневые признаки (края, градиенты яркости), а последующим агрегировать их в более сложные пространственные паттерны (формы, контуры знаков).

После трёх блоков пространственное разрешение карт признаков сокращается с 48×48 до 6×6. Вместо стандартного Flatten применяется Global Average Pooling(дальше GAP), который усредняет каждую карту признаков до одного числа, сводя тензор 128×6×6 к вектору длиной 128. Это существенно сокращает число параметров классификатора и добавляет дополнительную инвариантность к пространственным сдвигам. Классификатор состоит из двух полносвязных слоёв (128→256→количество классов) с ReLU и Dropout(0.5) между ними.

Выбор именно трёх блоков и данного числа каналов обусловлен компромиссом между вычислительной сложностью и выразительной способностью модели при входном разрешении 48×48: большее число блоков

привело бы к чрезмерному уменьшению пространственного разрешения ещё до GAP, тогда как меньшее — к недостаточной глубине иерархии признаков. Архитектура может быть изменена в нескольких направлениях: увеличение числа каналов в блоках (например, до 64→128→256) повысит выразительность модели ценой роста числа параметров и потребления памяти; добавление четвёртого блока потребует увеличения входного разрешения; замена MaxPool на свёртку с шагом 2 позволит модели самостоятельно обучать функцию субдискретизации. Наконец, значения Dropout (0.25 в блоках и 0.5 в классификаторе) подобраны эмпирически — их уменьшение ускорит обучение, но может привести к переобучению на несбалансированных классах датасета RTSD.

Листинг 1 — Структура модели

```
# Блок 1: 3 → 32 каналов, 48×48 → 24×24
block1 = Sequential(
    Conv2d(3, 32, kernel_size=3, padding=1),
    BatchNorm2d(32), ReLU(),
    Conv2d(32, 32, kernel_size=3, padding=1),
    BatchNorm2d(32), ReLU(),
    MaxPool2d(2, 2), Dropout2d(0.25)
)
# Блок 2: 32 → 64 каналов, 24×24 → 12×12
block2 = Sequential(
    Conv2d(32, 64, kernel_size=3, padding=1),
    BatchNorm2d(64), ReLU(),
    Conv2d(64, 64, kernel_size=3, padding=1),
    BatchNorm2d(64), ReLU(),
    MaxPool2d(2, 2), Dropout2d(0.25)
)
# Блок 3: 64 → 128 каналов, 12×12 → 6×6
block3 = Sequential(
    Conv2d(64, 128, kernel_size=3, padding=1),
    BatchNorm2d(128), ReLU(),
    Conv2d(128, 128, kernel_size=3, padding=1),
    BatchNorm2d(128), ReLU(),
    MaxPool2d(2, 2), Dropout2d(0.25)
)
gap = AdaptiveAvgPool2d(1)
classifier = Sequential(
    Flatten(),
    Linear(128, 256), ReLU(), Dropout(0.5),
    Linear(256, num_classes)
)
```

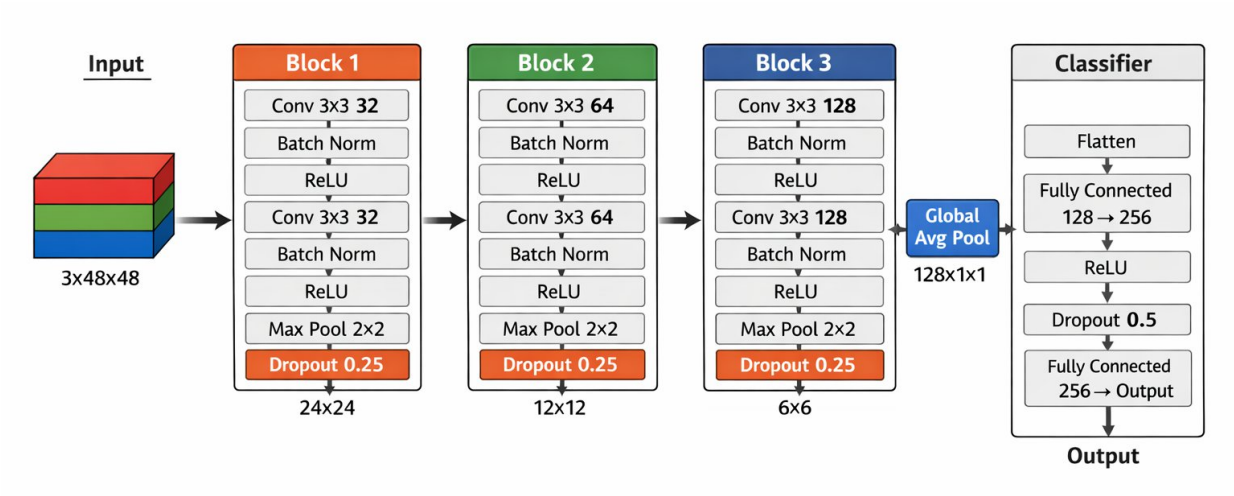

Рис. 1 Схема архитектуры нейронной сети.

2.2. Подготовка данных

Датасет RTSD загружается с платформы Hugging Face и содержит только одну общую выборку, поэтому она вручную делится в соотношении 90/10 на обучающую и тестовую части с фиксированным случайным зерном для воспроизводимости результатов. Поскольку исходные числовые метки классов в датасете не идут подряд с нуля, реализовано переотображение в плотный диапазон [0, N), где N — итоговое число классов.

Все изображения приводятся к единому размеру 48×48 пикселей и нормализуются с использованием статистик ImageNet. Для тестовой выборки применяется только эта нормализация. Набор аугментаций для обучающей выборки (случайные повороты, изменение яркости, горизонтальное отражение) предусмотрен в коде, их влияние исследуется отдельно в разделе анализа устойчивости модели.

Подача данных организована через стандартный PyTorch DataLoader с размером батча 64 изображения.

Рис. 2 Примеры изображений из датасета RTSD.

2.3. Алгоритм обучения

Обучение модели представляет собой итерационный процесс минимизации функции потерь на обучающей выборке путём последовательного обновления весов сети. На каждой итерации выполняется прямой проход и вычисление предсказания модели для текущего батча, затем вычисляется значение функции потерь, после чего методом обратного распространения ошибки находятся градиенты, и оптимизатор обновляет параметры сети. Ниже описаны конкретные компоненты этой схемы: выбор функции потерь, стратегия управления скоростью обучения и алгоритм вычисления градиентов.

2.3.1. Функция потерь и оптимизатор

В качестве функции потерь используется кросс-энтропия (CrossEntropyLoss). Для многоклассовой классификации она вычисляет расстояние между предсказанным распределением вероятностей и истинной меткой класса: чем увереннее модель предсказывает правильный класс, тем меньше значение потерь. В реализации PyTorch функция nn.CrossEntropyLoss совмещает операции Softmax и вычисления кросс-энтропии в одном численно устойчивом шаге, что исключает проблему исчезающего градиента при работе с логитами.

Для обновления весов применяется оптимизатор Adam. В отличие от классического стохастического градиентного спуска, Adam адаптирует скорость обучения индивидуально для каждого параметра, используя скользящие средние первого и второго моментов градиента. Начальное значение скорости обучения установлено равным lr = 1×10^{-3}.

2.3.2. Планировщик learning rate

По мере обучения фиксированная скорость обучения становится неоптимальной: слишком большое значение мешает точной сходимости, а слишком малое замедляет прогресс. Для решения этой проблемы применяется планировщик Cosine Annealing, при котором скорость обучения монотонно убывает от η_{max} до η_{min} по косинусному закону на протяжении T_{max} эпох. В работе используются параметры T_{max} = 50 эпох и $\eta_{min} = 1\times10^{-6}$. Такой режим позволяет модели делать крупные шаги в начале обучения и плавно «досходиться» к минимуму в конце, снижая риск застревания в локальных минимумах.

2.3.3. Обратное распространение ошибки

После вычисления функции потерь на текущем батче необходимо определить, как каждый параметр сети повлиял на итоговую ошибку. Это достигается методом обратного распространения ошибки: градиент функции потерь последовательно распространяется от выходного слоя к входному через рекурсивное применение цепного правила дифференцирования по всем узлам вычислительного графа. PyTorch строит этот граф автоматически в ходе прямого прохода и вычисляет все градиенты вызовом loss.backward(). Полученные градиенты передаются оптимизатору Adam, который производит обновление весов модели.

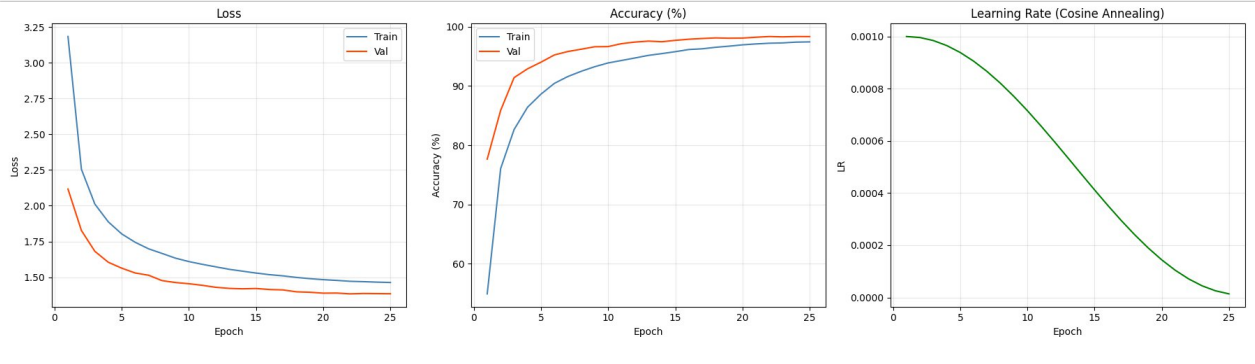

Рис. 3 Кривые обучения:Loss, Accuracy и learning rate по эпохам (train / val)

2.4. Исследование устойчивости к искажениям

Для оценки робастности модели тестовая выборка дополняется искажёнными версиями изображений трёх типов: гауссовский шум (σ = 0.05, 0.1, 0.2), размытие (радиус = 1, 3, 5) и окклюзия — случайное перекрытие части знака (10–50% площади). Каждый тип моделирует реальные условия эксплуатации: шум возникает при съёмке в условиях низкой освещённости, размытие — при движении или тряске камеры, окклюзия — при частичном перекрытии знака посторонними объектами. По результатам измерения аккуратности на каждой версии строится «профиль устойчивости» модели, а дополнительное сравнение моделей, обученных без аугментаций и с ними, позволяет оценить практическую ценность аугментации.

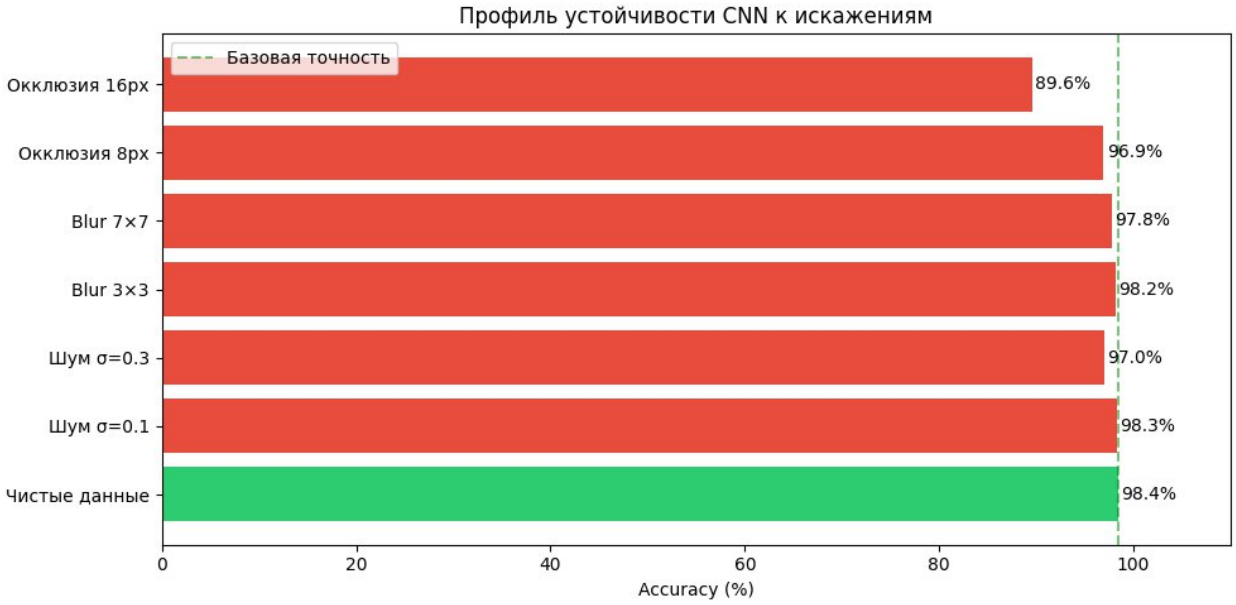

Рис. 4 Профиль устойчивости: зависимость Accuracy от типа и интенсивности искажения

2.5. Визуализация ошибок

Высокая точность модели на чистых данных не гарантирует корректности её внутренней логики: модель может случайно «угадывать» правильный класс, опираясь на нерелевантные признаки например, фон, артефакты съёмки или случайные текстуры. Для выявления подобных проблем применяются методы интерпретируемости, позволяющие визуализировать, какие именно области входного изображения оказали наибольшее влияние на решение модели.

Первый метод — Grad-CAM (Gradient-weighted Class Activation Mapping). Его суть состоит в следующем: для целевого класса вычисляются градиенты функции скора относительно карт признаков (feature maps) последнего сверточного слоя. Эти градиенты усредняются по пространственным позициям и используются как веса для взвешенного суммирования карт признаков. Результат низкоразрешённая тепловая карта, которая затем билинейно интерполируется до размера входного изображения и накладывается на него. Области с высокой активацией соответствуют тем частям изображения, которые модель сочла наиболее значимыми при классификации. Метод применяется именно к последнему сверточному слою, поскольку он содержит наиболее семантически насыщенные признаки при наибольшем пространственном разрешении, сохранившемся после пулинга.

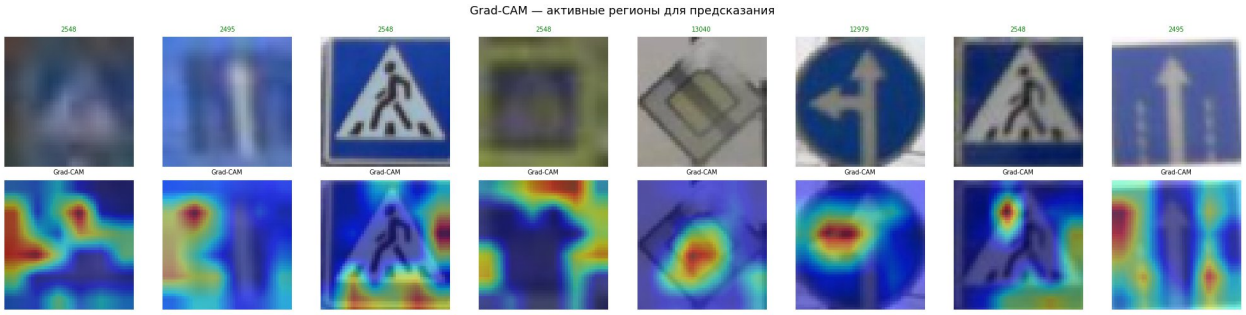

Рис. 5 Визуализация с помощью Grad-CAM.

Второй метод — Saliency Map (карта значимости). В отличие от Grad-CAM, он работает не на уровне внутренних карт признаков, а напрямую с пикселями входного изображения: вычисляется градиент скора

предсказанного класса по каждому пикселю входа. Большое по модулю значение градиента означает, что незначительное изменение данного пикселя существенно меняет предсказание модели — то есть этот пиксель «важен» для решения. Карта значимости даёт более детальную, попиксельную картину внимания модели, однако зачастую более зашумлённую по сравнению с Grad-CAM.

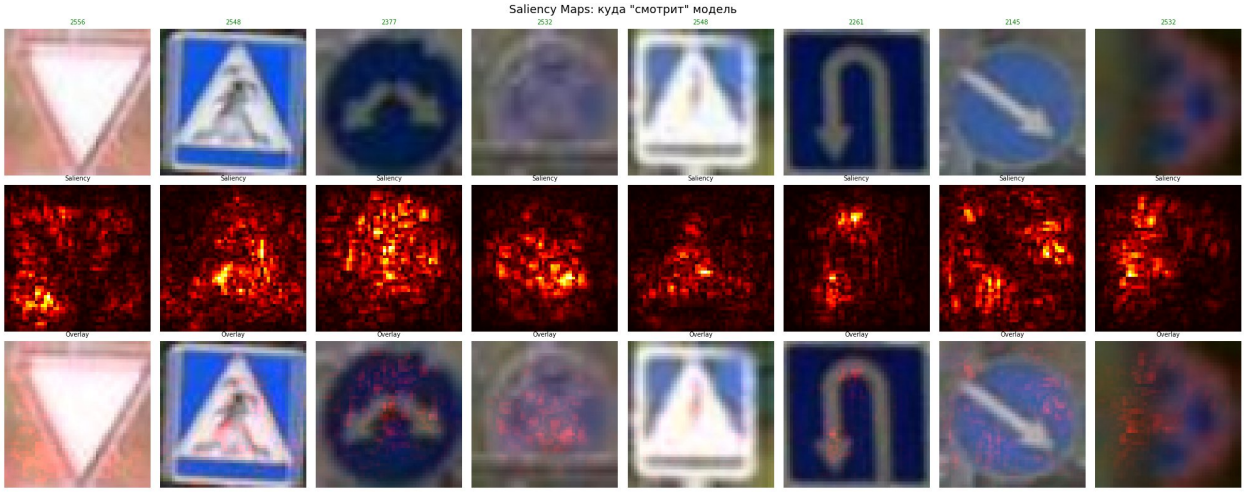

Рис. 6 Визуалиция с помщью Saliency Map.

Оба метода применяются к примерам, на которых модель совершает ошибку. Совместный анализ тепловых карт позволяет выявить систематические паттерны неверной классификации: например, смещение внимания на фоновые объекты, блики или элементы дорожной разметки вместо самого знака. На основе этих наблюдений формулируются выводы о возможных улучшениях — как в части архитектуры, так и в части стратегии аугментации обучающей выборки.

3. АНАЛИЗ ВЫЧИСЛИТЕЛЬНОЙ СЛОЖНОСТИ

3.1. Теоретический подсчёт FLOPs

Оценка вычислительной сложности проводится в терминах числа операций с плавающей точкой. Принято соглашение: 1 MAC (умножение + сложение) = 2 FLOPs.

Формула для сверточного слоя:

$$FLOPs_{conv} = 2 \cdot H_{out} \cdot W_{out} \cdot C_{out} \cdot C_{in} \cdot K^2$$

где C_{in}, C_{out} — число входных и выходных каналов; K — размер ядра; H_{out}, W_{out} — размеры выходной карты признаков.

Формула для полносвязного слоя:

$$FLOPs_{linear} = \boldsymbol{d_{in}} \cdot \boldsymbol{d_{out}}$$

Таблица 1 — Поэтапный подсчёт FLOPs по слоям модели

Слой	FLOPs	Тип
Conv1 (3→32, 48×48)	1 990 656	свёрточный
Conv2 (32→32, 48×48)	21 233 664	свёрточный
BatchNorm Блок 1 (2 сл.)	294 912	нормализация
Conv3 (32→64, 24×24)	10 616 832	свёрточный
Conv4 (64→64, 24×24)	21 233 664	свёрточный
BatchNorm Блок 2 (2 сл.)	147 456	нормализация
Conv5 (64→128, 12×12)	10 616 832	свёрточный
Conv6 (128→128, 12×12)	21 233 664	свёрточный
BatchNorm Блок 3 (2 сл.)	73 728	нормализация
Global Average Pooling	4 608	пулинг
Linear 1 (128→256)	32 768	полносвязный

Слой	FLOPs	Тип
Linear 2 (256→15628)	4 000 768	полносвязный
ИТОГО	**182 959 104**	**—**

Итоговая теоретическая вычислительная сложность модели:

$$FLOPs_{total} = 182\,959\,104 \approx 182{,}96 \text{ млн } FLOPs$$

Значение верифицировано с помощью библиотеки fvcore и torchinfo. Операции MaxPool2d и Dropout не учитываются ввиду пренебрежимо малого вклада; основную нагрузку (более 95%) создают сверточные слои.

3.2. Практическая оценка времени инференса

Практические измерения проводились на Apple MacBook Air M2 (8-ядерный GPU, заявленная производительность 2,84 TFLOPs). Модель запускалась с различными размерами батча (1, 4, 8, 16, 32, 64, 128, 256). Теоретическое время инференса:

$$T_{theory} = FLOPs_{forward} \cdot \frac{N}{GPU_{TFLOPs}}$$

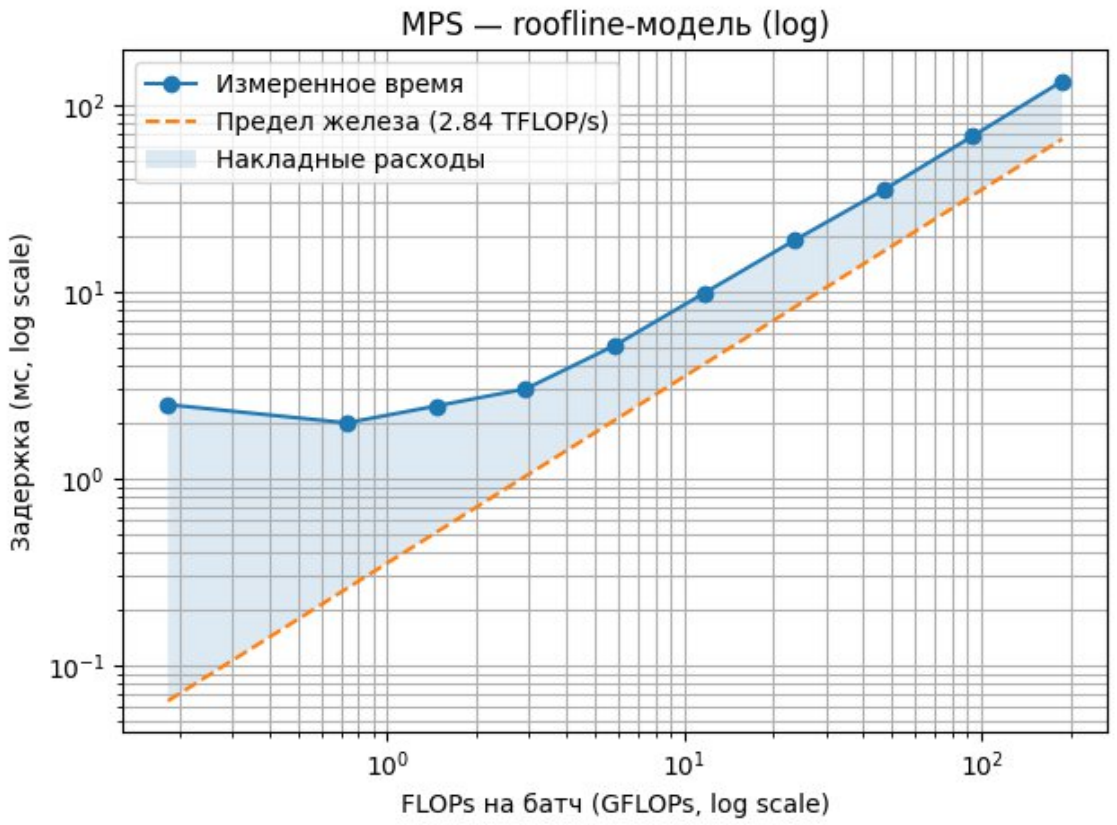

Рис. 7 Зависимость времени инференса (GPU metal) от batch size: теория (оранжевый график) и реальность (синий)

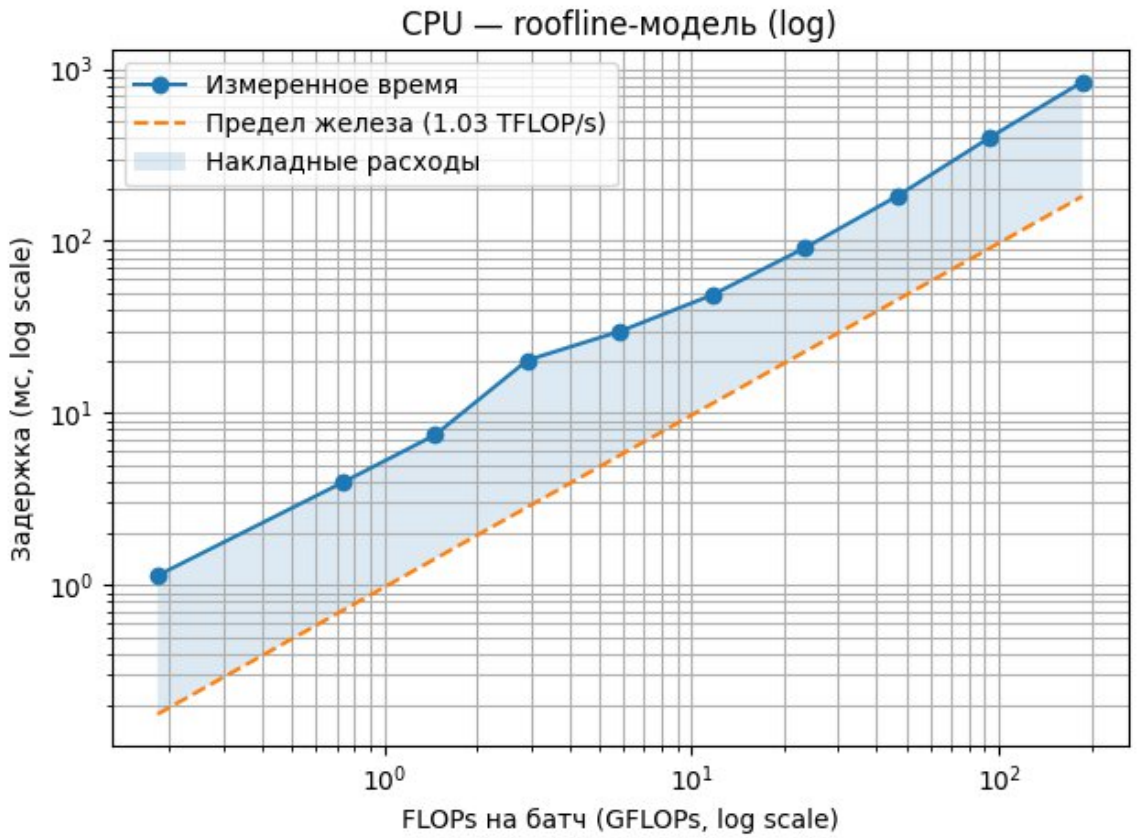

Рис. 8 Зависимость времени инференса (CPU) от batch size: теория (оранжевый график) и реальность (синий)

Расхождение теоретических и фактических значений обусловлено накладными расходами ОС, задержками аллокатора памяти и «холодным стартом» GPU на малых батчах. Реально достигнутая эффективность GPU составила около 49% от заявленной мощности. На батче размером 1 инференс оказался эффективнее на CPU (0.05 мс против 0.52 мс у GPU), так как накладные расходы инициализации занимают значительную долю времени на маленькой модели.

3.3. Анализ потребления памяти

Теоретический объём памяти для хранения весов в формате float32 (4 байта):

$$4\ 337\ 324 \times 4 \approx 16{,}55\ \text{MiB}$$

С учётом входного изображения 3×48×48:

$$16{,}55\ \text{MiB} + 27\ 648\ \text{байт} \approx 16{,}57\ \text{MiB}$$

Верификация через PyTorch API показала: вес модели 16.5490 MiB, с входом 16.5754 MiB — результаты совпадают с теорией.

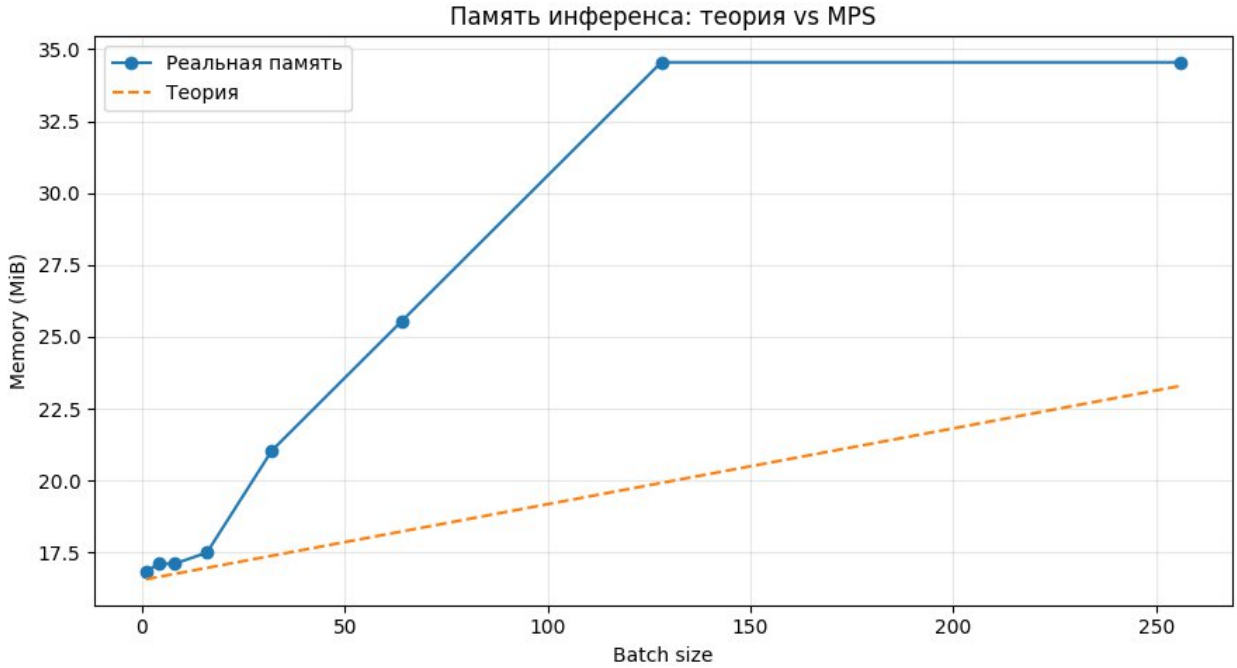

Рис. 9 Реальное и теоретическое потребление памяти GPU metal

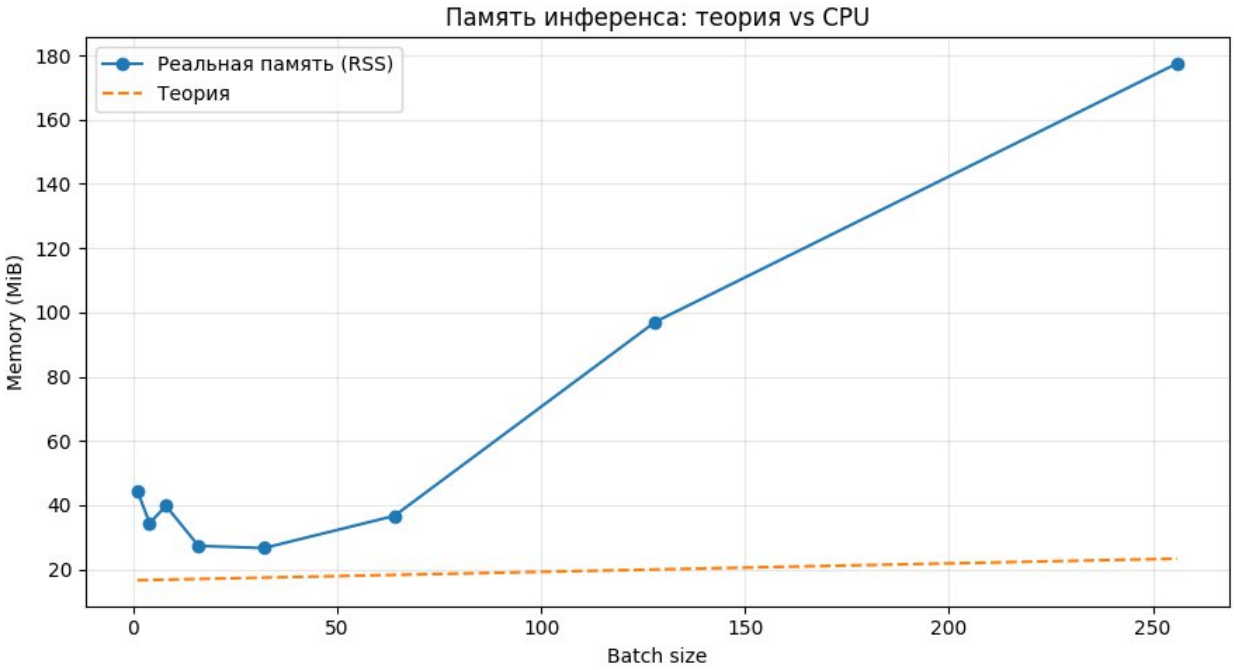

Рис. 10 Реальное и теоретическое потребление памяти CPU

Превышение реального потребления над теоретическим на крупных батчах объясняется промежуточными тензорами активаций, рабочими буферами алгоритмов свёртки и оверхедом аллокатора памяти. Теоретическая оценка соответствует нижней границе потребления.

4. ТЕСТИРОВАНИЕ

Загрузим в модель разные изображения и проверим как она их оценивает:

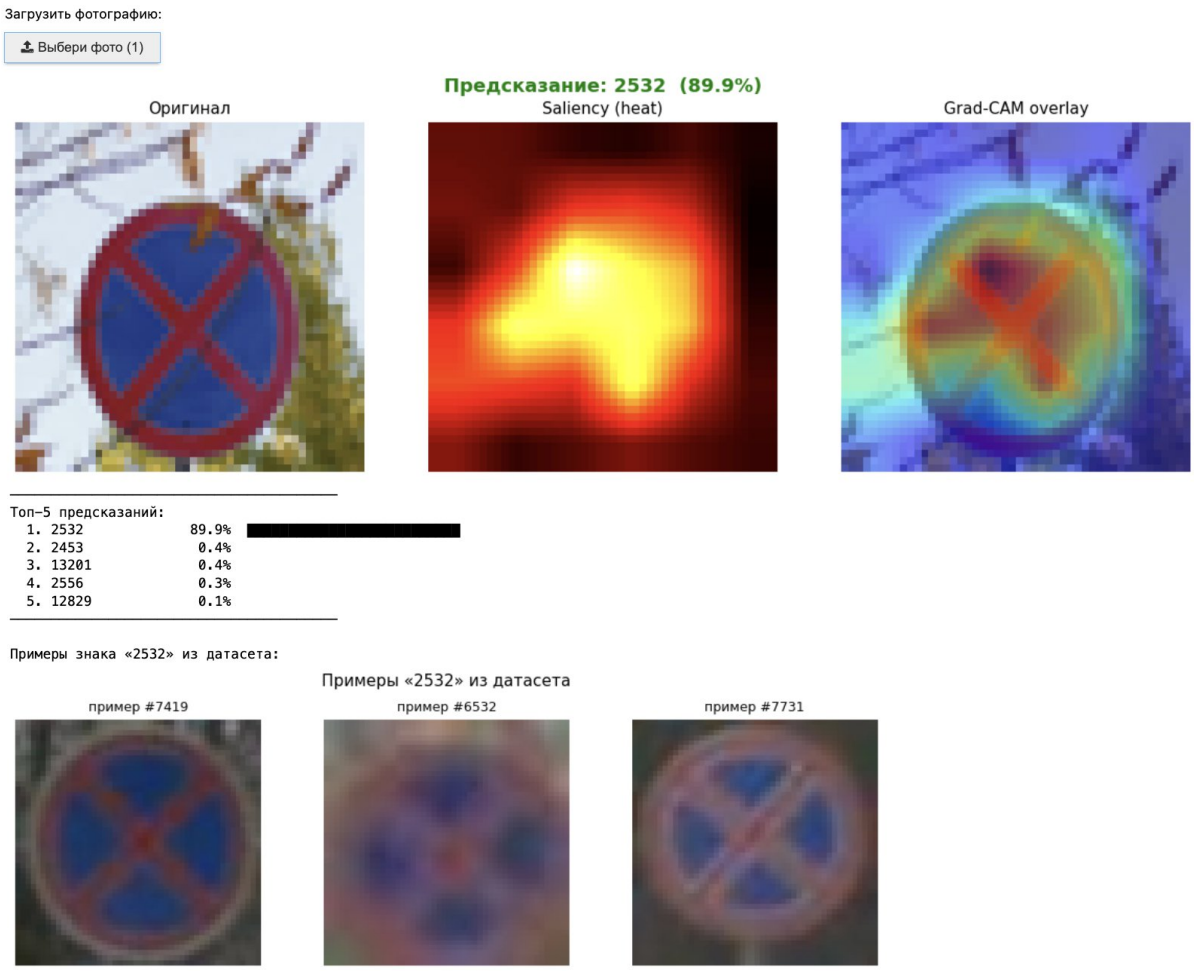

Рис. 11 Тестирование модели на случайном изображение.

Также попробуем загрузить что то более сложное, где знак не занимает большую часть фотографии:

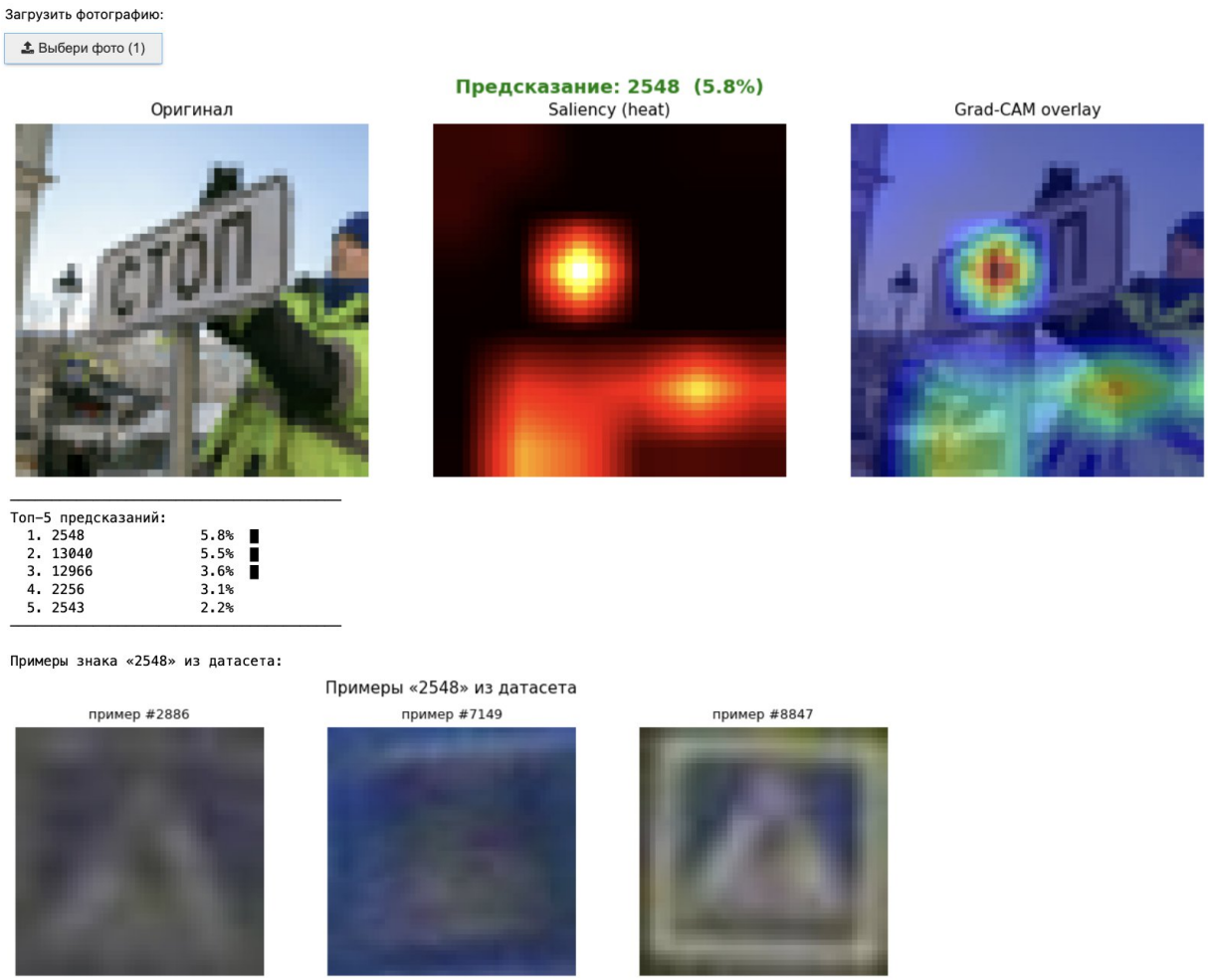

Рис. 12 Неудачное оценивание знака моделью.

Как видно по последнему тесту Рис. 12 модель не увидела тот знак, который реально был на фото, это можно связать с низким разрешением изображения и с тем что сам знак не занимает большую часть фотографии, как это было на примерах датасете для обучения.

5. ИТОГИ ПРОДЕЛАННОЙ РАБОТЫ

В ходе выполнения курсового проекта разработана и исследована свёрточная нейронная сеть для классификации дорожных знаков из набора данных RTSD. Реализована трёхблочная архитектура с пакетной нормализацией, субдискретизацией, послойным прореживанием и глобальным усредняющим пулингом на основе фреймворка PyTorch.

Проведена теоретическая оценка вычислительной сложности модели — 182,96 млн операций с плавающей точкой на одно изображение, — верифицированная с помощью библиотеки fvcore. Теоретические оценки потребления памяти (≈16,57 МиБ) подтверждены измерениями средствами PyTorch.

Практические измерения времени вывода на графическом и центральном процессорах устройства Apple M2 (8 GPU и 4+4 CPU ядер) показали, что фактически достигаемая вычислительная эффективность составляет около 49% и 25% от заявленной пиковой производительности соответственно. Основные источники расхождения — накладные расходы операционной системы, управление памятью и задержки при первом запуске на графическом процессоре.

Исследование устойчивости показало, что обучение с применением искусственного расширения обучающей выборки существенно повышает надёжность модели при наличии шума, размытия и частичного перекрытия знаков. Анализ ошибок с помощью тепловых карт Grad-CAM позволил выявить систематические паттерны, связанные со смещением внимания модели на фоновые элементы изображения.

В перспективе проект допускает развитие по нескольким направлениям. С точки зрения архитектуры представляет интерес переход к более глубоким моделям с остаточными связями (по аналогии с семейством ResNet): такие связи позволяют градиенту свободно распространяться через большое число слоёв, что открывает возможность наращивать глубину сети

без деградации качества обучения. Для повышения устойчивости при работе с изображениями высокого разрешения целесообразно увеличить входной размер с 48×48 до 96×96 или 128×128 и добавить четвёртый сверточный блок — это позволит модели улавливать более мелкие детали знаков, существенные при классификации визуально схожих категорий. Альтернативным направлением является применение механизма пространственного внимания (spatial attention), который позволяет модели явно сосредоточиться на области знака, подавляя влияние фона, что непосредственно адресует проблему, выявленную в ходе тестирования. Отдельный интерес представляет использование предобученных моделей с переносом знаний: дообучение на датасете RTSD сети, предварительно обученной на крупной общей выборке, способно существенно повысить точность при ограниченном объёме размеченных данных. Наконец, с практической точки зрения перспективным является сжатие модели методами прореживания весов или квантования, что позволит развернуть её непосредственно на бортовом вычислителе с ограниченными ресурсами без значительной потери качества распознавания.

СПИСОК ЛИТЕРАТУРЫ

1. LeCun, Y., Boser, B., Denker, J. S. et al. Back-Propagation Applied to Handwritten Zip Code Recognition // Neural Computation. — 1989. — Vol. 1, No. 4. — P. 541–551.
2. Lee, H., Grosse, R., Ranganath, R., Ng, A. Y. Unsupervised Learning of Hierarchical Representations with Convolutional Deep Belief Networks // Communications of the ACM. — 2011. — Vol. 54, No. 10. — P. 95–103.
3. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R. Dropout: A Simple Way to Prevent Neural Networks from Overfitting // Journal of Machine Learning Research. — 2014. — Vol. 15. — P. 1929–1958.
4. Loshchilov, I., Hutter, F. SGDR: Stochastic Gradient Descent with Warm Restarts // ICLR 2017. — [Электронный ресурс]. — URL: https://arxiv.org/abs/1608.03983.
5. Selvaraju, R. R. et al. Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization // Proceedings of ICCV. — 2017. — P. 618–626.
6. PyTorch Documentation. — [Электронный ресурс]. — URL: https://docs.pytorch.org/ (дата обращения: 09.04.2026).
7. RTSD Dataset (cleaned). — [Электронный ресурс]. — URL: https://huggingface.co/datasets/eleldar/rtsd_cleaned (дата обращения: 09.04.2026).
8. fvcore: FlopCountAnalysis. — [Электронный ресурс]. — URL: https://detectron2.readthedocs.io/en/stable/modules/fvcore.html (дата обращения: 09.04.2026).
9. Utkuozbulak. PyTorch CNN Visualizations. — [Электронный ресурс]. — URL: https://github.com/utkuozbulak/pytorch-cnn-visualizations (дата обращения: 09.04.2026).